

ICS EXE 5

2026.04.03

Problem 1

What is the output of following code?

```
1. int main() {
2.     if (fork() && (!fork())) {
3.         if (fork() || fork()) {
4.             fork();
5.         }
6.     }
7.     printf("3 ");
8.     while(waitpid(-1, NULL, 0) > 0);
9.     return 0;
10. }
```

Problem 2

```
1. #include "csapp.h"
2. volatile int sig_user1_cnt = 0;
3. int child_exit_num = 0;
4. int num = 3;
5. int status = 40;
6. pid_t pid[3];
7.
8. void handle_siguser1(int num) { sig_user1_cnt++; }
9.
10. int main() {
11.     signal(SIGUSR1, handle_siguser1);
12.     for (int i = 0; i < 3; i++) {
13.         status++;
14.         if ((pid[i] = Fork()) == 0) { // Child Process
15.             num++;
16.             printf("num=%d\n", num); fflush(stdout);
17.             kill(getppid(), SIGUSR1);
18.             exit(status);
19.         } else // Parent Process
20.             num += 10;
21.     }
```

```

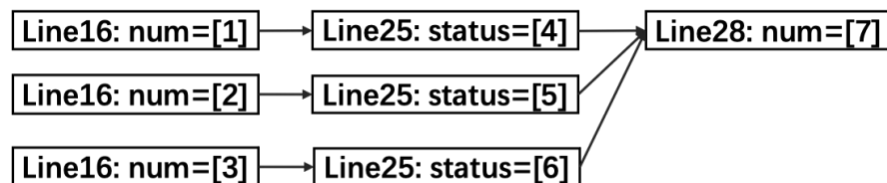
22.     while (waitpid(-1, &status, 0) > 0) {
23.         child_exit_num++;
24.         if (WIFEXITED(status)) {
25.             printf("status=%d\n", WEXITSTATUS(status)); fflush(stdout);
26.         }
27.     }
28.     printf("num=%d\n", num); fflush(stdout);
29.     printf("sig_user1_cnt=%d\n", sig_user1_cnt); fflush(stdout);
30.     return 0;
31. }
32.

```

Assumptions:

- We assume OS may stop and resume the execution of each process arbitrarily.
- The kernel checks the pending bits when it switches from kernel to user (e.g., returning from system call). The return of signal handler will trigger *sigreturn* system call.

1. How many times will **fork()** be called in this program.
2. Will all the forked child processes be reaped by parent process finally? Why?
3. The following figure describes parts of contents printed on the screen by **Line 16**, **Line 25** and **Line 28**. In this figure, “**OutputX** -> **OutputY**” indicates that **OutputX** must appear before **OutputY** on the screen. You should fill the blanks, and make sure that your answer matches the order of each output.



4. The child process may terminate in any order. Show how to modify the codes at **Line 22** to make the **value** of **status** at **Line 25** printed at screen in **increasing order**. (Hint: You can refer to **Figure 8.19** in your textbook)
5. Show the **maximum value** and **minimum value** of **sig_user1_cnt** at **Line 29**. For each value, show a possible execution order processes may execute in.